에코 루프 기획서 v2: 게임 + 릴스 성장

Oct 21, 2026 · @LEE seongjun

v2 목표: 6주 안에 폰에서 바로 해볼 수 있는 웹 MVP를 내고, 개발 과정과 플레이를 릴스 시리즈로 올려 팔로워를 모은다. 모든 기능은 "이게 릴스 3초 안에 보이나?"를 같이 따진다.

개요

가제: 에코 루프(Echo Loop) — 10초 동안의 내 행동이 녹화되고, 다음 루프마다 과거의 내가 그대로 반복한다. 과거의 나들과 협력해 한 화면짜리 방을 탈출하는 2D 퍼즐 플랫포머.

- 장르: 2D 퍼즐 플랫포머, 싱글 플레이
- 엔진: Godot 4 / GDScript, 웹(HTML5) 배포 우선
- 분량: MVP 15스테이지, 플레이 타임 40~60분
- 목표: 4주 안에 itch.io에 올릴 수 있는 MVP 완성

핵심 재미는 "혼자 하는 협동"이다. 방을 보고 "1번째 나는 버튼을 누르고, 2번째 나는 발판이 되고, 지금의 나는 출구로" 같은 역할 분담을 계획한다. 그 계획이 한 번에 맞물려 돌아가는 순간이 이 게임의 보상이다.

레퍼런스	비슷한 점	에코 루프의 차별점
Braid	시간을 다루는 퍼즐	되돌리기가 아니라 쌓기: 과거의 내가 사라지지 않고 계속 협력한다
Cursor*10	이전 시도가 복제되어 함께 움직임	플랫포머 물리: 과거의 나를 밟고 올라서거나 레이저를 몸으로 막을 수 있다

페르소나 리뷰 (v1 → v2)

v1은 "재밌는 퍼즐 게임"까지만 설계돼 있었다. 6명의 관점에서 보면 화면 비율, 실행 난이도, 업로드 공백 세 가지가 가장 큰 구멍이었다.

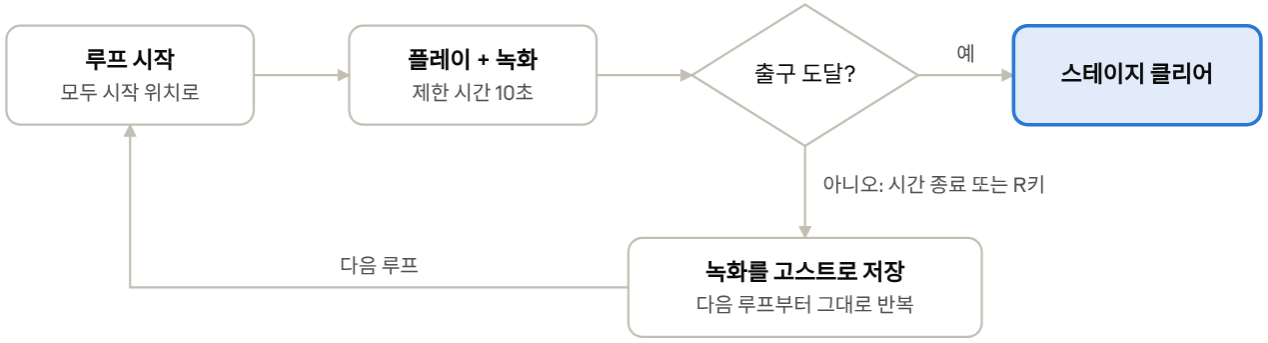
페르소나	원하는 것	v1의 구멍	v2에서 바꾼 것
릴스 스크롤러 (게임 모름, 엄지로 넘김)	1초 안에 "뭐지?", 3초 안에 규칙 이해	가로 화면이라 폰에서 캐릭터가 점처럼 보임. 규칙을 글로 설명해야 함	세로 9:16 방, 고스트 번호·색 구분, 큰 루프 카운터, 모든 나를 겹쳐 보여주는 클리어 리플레이

페르소나	원하는 것	v1의 구멍	v2에서 바꾼 것
퍼즐 게이머 (Baba Is You, Portal 좋아함)	손 실력보다 "아하" 하는 순간	10초 안에 정확한 점프를 요구해 실패가 손 문제가 됨. 고스트 하나 틀리면 처음부터	고스트 개별 재녹화, 루프 타임라인, 코요테 타임·점프 버퍼, 최소 고스트 수 별점
AI·개발 콘텐츠 팔로워 (바이브코딩 관심 대학생)	어떻게 만들었는지, 따라 할 수 있는 것	개발 과정이 콘텐츠로 설계돼 있지 않음	Claude와 고닷으로 만드는 과정을 주 1회 개발 일지로, 핵심 코드는 댓글 키워드로 배포
인스타 알고리즘 (시청 지속·저장·공유·댓글)	첫 3초 이탈 없음, 반복 시청, 꾸준한 시리즈	완성 후 올리는 계획이라 개발 기간 내내 업로드 공백	1주차부터 공개 개발, 주 3회 업로드, 끝이 처음으로 이어지는 루프 편집, 문제편→정답편 2부작
릴스 보고 들어온 첫 플레이어 (폰으로 링크 누름)	설치 없이 폰에서 바로, 30초 안에 재미	키보드 전용, 첫 방에 고스트가 안 나옴	모바일 터치 버튼을 MVP에 포함, 1-1에서 바로 과거의 나를 밟고 탈출
1인 개발자 본인 (학기 중, 수·금 저녁 알바)	학기 중에 끝낼 수 있는 범위	촬영·편집 시간이 일정에 없음	6주로 늘림, 촬영은 고닷 Movie Maker로 자동화, 편집은 기존 insta_auto 재사용, 12스테이지 컷 기준

핵심 루프와 규칙

한 루프는 10초, 루프가 끝날 때마다 방이 초기화되고 방금의 나는 고스트로 남는다. 출구에 닿을 수 있는 건 항상 "지금의 나" 한 명뿐이다.

출구에 닿을 때까지 루프마다 과거의 내가 한 명씩 늘어난다



핵심 루프 · 4단계, 분기 1개

기본 규칙

1. 스테이지는 한 화면짜리 방이다. 루프 길이는 기본 10초, 스테이지마다 6~15초로 조정한다.
2. 루프는 제한 시간이 끝나거나 R키를 누르면 끝난다. 방 전체(문, 발판, 상자)가 초기 상태로 돌아가고 방금 플레이가 고스트로 저장된다.
3. 스테이지마다 고스트 상한(1~4명)이 있다. 상한이 차면 더 저장되지 않는다.
4. 클리어 조건은 지금의 내가 출구에 닿는 것이다. 고스트는 출구를 쓸 수 없다.
5. 고스트는 실체가 있다. 밟고 올라설 수 있고, 압력판을 누르고, 레이저를 몸으로 막는다.
6. Backspace는 마지막 고스트 삭제, Shift+R은 스테이지 처음부터 다시.

v2 추가 규칙 (퍼즐 게이머 피드백)

- **고스트 개별 재녹화**: 숫자키 1~4(모바일은 고스트 아이콘 탭)로 "다음 루프에서 이 번호의 나를 덮어쓰기"를 고른다. 2번만 실수했으면 2번만 다시 찍는다. 막히는 게 손 문제가 아니라 머리 문제가 되게 하는 핵심 장치다.
- **루프 타임라인**: 화면 하단에 고스트별 가로 줄을 깔고, 각자 점프·상호작용한 시점을 점으로 찍는다. "1번이 4초에 버튼을 누른다"를 눈으로 계획할 수 있다.
- **조작 보정**: 코요테 타임 0.1초, 점프 버퍼 0.1초. 루프가 끝나는 순간 누르고 있던 키는 다음 루프로 넘기지 않는다(리셋 직후 오입력 방지).
- **루프 카운터**: "LOOP 3 · 나 ×3"처럼 지금 몇 번째 루프인지 크게 표시한다. 릴스 시청자가 규칙을 숫자 하나로 이해한다.

녹화 방식: 입력이 아니라 위치를 녹화한다

- 키 입력을 녹화해서 재생하면, 환경이 조금만 달라져도 고스트가 엉뚱하게 움직여 버그처럼 보인다.
- 그래서 매 물리 프레임의 위치와 상태(애니메이션, 상호작용 여부)를 녹화하고, 고스트는 그 경로를 그대로 따라간다.
- 대신 고스트의 경로가 막히면(닫힌 문, 움직인 지형) 그 고스트는 **패러독스로 붕괴**한다. 글리치 효과와 함께 사라지고, 그 루프에서는 더 이상 움직이지 않는다.
- 이 규칙 자체가 퍼즐 제약이 된다: "과거의 나를 방해하지 마라."

레벨 오브젝트와 퍼즐 요소

오브젝트는 8종으로 시작한다. 모두 "신호를 보내는 것"과 "신호를 받는 것"으로 나뉘어서, 하나의 시그널 구조로 연결할 수 있다.

오브젝트	동작	퍼즐에서의 역할	첫 등장
압력판	누군가 올라가 있는 동안만 신호	고스트 한 명을 "문지기"로 세워두기	1-2
문	신호가 있으면 열림	닫힌 문은 고스트 경로를 막아 패러독스를 일으킴	1-2

오브젝트	동작	퍼즐에서의 역할	첫 등장
레버	한 번 당기면 루프 끝까지 신호 유지	누가, 언제 당길지 타이밍 설계	1-4
이동 발판	신호가 있는 동안 정해진 경로로 이동	고스트가 발판을 움직이고 지금의 내가 탄다	2-1
레이저	닿으면 탈락: 고스트는 붕괴, 지금의 나는 루프 즉시 종료(녹화 저장 안 함)	고스트를 몸으로 세워 레이저 막기	2-3
색 문	파랑은 고스트만, 주황은 지금의 나만 통과	역할 분리를 강제	2-5
시한 버튼	누르면 3초간 신호	여러 명이 연달아 누르는 릴레이	3-1
상자	밀 수 있고 압력판을 누를 수 있음, 루프마다 원위치	고스트가 옮겨둔 상자를 지금의 내가 발판으로 사용	3-3

상자는 고스트(고정 경로)와 물리 오브젝트(실시간 계산)가 부딪히는 유일한 요소라 구현 난이도가 가장 높다. MVP 일정이 밀리면 가장 먼저 빼는 후보다.

레벨 디자인

15스테이지, 3챕터 구성이다. 한 스테이지에 새 요소는 최대 하나만 넣고, 챕터 마지막 스테이지에서 그 챕터 요소를 전부 섞는다.

스테이지	이름	새 요소	고스트 상한	핵심 아이디어
1-1	첫 걸음	이동, 점프, 루프	0	시간 안에 출구 도착. 시간이 지나면 리셋된다는 것만 체험
1-2	문지기	압력판, 문	1	과거의 내가 압력판을 밟고 있는 동안 지나간다
1-3	디딤돌	고스트 밟기	1	높은 턱 아래 서 있는 과거의 나를 밟고 점프
1-4	두 개의 문	레버	1	레버는 내가, 압력판은 과거의 내가
1-5	막지 마라	패러독스	2	순서를 잘못 짜면 내가 과거의 나를 가둔다
2-1	엘리베이터	이동 발판	2	과거의 내가 발판을 움직이고 지금의 내가 탄다
2-2	탑 쌓기	고스트 위 고스트	2	과거의 나 둘을 쌓아 높은 곳으로
2-3	방패	레이저	2	과거의 나를 레이저 앞에 세워 길을 연다

스테이지	이름	새 요소	고스트 상한	핵심 아이디어
2-4	교대 근무	압력판 시간차	3	한 명이 떠나는 순간 다른 한 명이 이어 밟는다
2-5	파랑과 주황	색 문	3	고스트만 갈 수 있는 길과 나만 갈 수 있는 길
3-1	릴레이	시한 버튼	3	3초 신호를 끊기지 않게 세 명이 이어간다
3-2	6초	짧은 루프	3	루프가 짧아져 동선을 압축해야 한다
3-3	짐꾼	상자	3	과거의 내가 옮겨둔 상자가 지금의 내 발판
3-4	역순	순서 퍼즐	4	녹화 순서와 실제 필요한 순서가 반대
3-5	에코	종합	4	모든 요소를 쓰는 마지막 방

플레이 테스트 기준: 각 스테이지를 처음 보는 사람이 5분 안에 풀면 적정, 10분 넘게 막히면 힌트 문구나 레벨 배치를 고친다.

v2 레벨 변경

- **1-1 재설계 (첫 플레이어 피드백)**: 출구를 혼자서는 못 닿는 턱 위에 둔다. 첫 루프는 반드시 실패하고, 바로 생긴 과거의 나를 밟고 탈출한다. 링크로 들어온 사람이 30초 안에 게임의 핵심을 체험한다.
- **세로 방**: 모든 방을 세로 9:16(18×32 타일)으로 설계한다. 세로 방은 탑 쌓기, 엘리베이터, 낙하 같은 위아래 퍼즐과 잘 맞는다.
- **별점**: 클리어 ★1, 상한보다 적은 고스트로 클리어 ★2, 개발자 최소 기록과 같으면 ★3. "★3 기록 깬 사람?"이 그대로 챌린지 릴스 소재가 된다.
- **시그니처 스테이지**: 1-5 막지 마라(패러독스), 2-2 탑 쌓기, 3-1 릴레이, 3-5 에코. 클리어 순간이 화면 한 장으로 설명되는 방이라 연출과 리플레이를 여기에 가장 공들인다.
- **12스테이지 컷 기준**: 일정이 밀리면 1-4, 2-4, 3-4를 뺀다. 각 챕터의 도입·시그니처·종합 방은 남긴다.

조작, UI, 연출

조작은 키보드 6개로 끝낸다. 웹에서 바로 해볼 수 있게 마우스는 쓰지 않는다.

키	동작
A / D 또는 ← / →	이동
Space	점프
E	레버 당기기, 시한 버튼 누르기
R	루프 수동 종료 (지금 플레이를 고스트로 저장)

키	동작
Backspace	마지막 고스트 삭제
Shift + R	스테이지 처음부터 다시

추가 조작: 숫자키 1~4는 다음 루프에서 덮어쓸 고스트 선택. **모바일(MVP 포함)**: 화면 하단 왼쪽에 ← →, 오른쪽에 점프·E, 상단 고스트 아이콘 탭으로 재녹화 선택, 루프 종료는 타이머 바를 길게 누르기.

화면 비율: 세로 9:16. 릴스 화면을 그대로 게임 화면으로 쓴다. 따로 크롭하거나 편집하지 않아도 캐릭터가 크게 보인다. PC 웹에서는 가운데 세로 화면 + 양옆에 조작법·타임라인 패널을 둔다.

HUD

- 화면 상단: 루프 남은 시간 바. 마지막 3초는 색이 바뀌고 똑딱 소리
- 좌상단: 고스트 슬롯 아이콘 (예: ●●○ = 3명 중 2명 사용)
- 고스트 머리 위: 번호(1, 2, 3)와 고유 색

연출 (작지만 체감이 큰 것부터)

- 루프 리셋: 0.5초 동안 화면이 역재생되듯 되감기는 효과 + 테이프 되감는 소리. 이 게임의 시그니처라 가장 먼저 다듬는다
- 고스트: 반투명 + 약한 잔상, 다음 루프에 할 동작을 점선 경로로 미리 보여주는 옵션
- 패러독스 붕괴: 고스트가 지직거리며 흩어지는 글리치 셰이더 + 짧은 경고음
- 클리어: 그 스테이지에서 쓴 모든 루프를 겹쳐서 2배속으로 다시 보여주는 리플레이. 짧은 영상으로 공유하기 좋다

아트 방향: 어두운 배경 + 단색 도형 위주의 미니멀 스타일. 캐릭터는 지금의 나만 흰색, 고스트는 각기 다른 색이라 누가 누군지 한눈에 구분된다. 에셋 부담이 적어서 1인 개발에 맞다.

릴스용 연출 원칙: 고스트 색은 4가지 고정(1 파랑, 2 주황, 3 초록, 4 보라)으로 시리즈 내내 바꾸지 않는다. 클리어 리플레이는 모든 루프를 겹쳐서 시작하고, 마지막 프레임이 루프 리셋 장면으로 끝나 영상 처음과 자연스럽게 이어진다. 되감기 효과음은 시그니처 사운드로 고정해서, 소리만 들어도 이 시리즈인 걸 알게 한다.

고닷 기술 설계

핵심은 세 파일이다: 녹화하는 `player.gd`, 재생하는 `ghost.gd`, 루프를 돌리는 `loop_manager.gd`. 나머지 오브젝트는 전부 `reset()` 하나만 구현하면 된다.

씬 구조

```
Main (Node)
├─ LoopManager (Node)          # 틱 카운트, 루프 종료, 고스트 생성
├─ Level (Node2D)               # 스테이지마다 교체 (level_1_2.tscn 등)
|  ├─ TileMapLayer              # 벽·바닥
|  ├─ SpawnPoint (Marker2D)
|  ├─ Exit (Area2D)
|  └─ Objects (Node2D)          # 압력판, 문, 레버… 각각 별도 씬
├─ Player (CharacterBody2D)     # 지금의 나
├─ Ghosts (Node2D)              # Ghost 인스턴스들
└─ HUD (CanvasLayer)
```

충돌 레이어: 1 = 지형(벽, 문), 2 = 플레이어, 3 = 고스트, 4 = 센서(압력판, 레이저, 출구). 고스트의 마스크는 지형만 켜서, 패러독스 검사가 지형 변화에만 반응하게 한다. 플레이어 마스크는 지형 + 고스트라서 고스트를 밟고 설 수 있다.

녹화 (player.gd): 물리 프레임마다 위치와 상호작용 여부를 저장한다. 60틱 기준 10초 루프 = 600프레임이라 메모리 걱정은 없다.

```
# player.gd
extends CharacterBody2D

var recording: Array[Dictionary] = []

func _physics_process(delta: float) -> void:
    # ... 이동, 점프 처리 ...
    move_and_slide()
    recording.append({
        "pos": global_position,
        "flip": $Sprite2D.flip_h,
        "interact": Input.is_action_just_pressed("interact"),
    })

func reset() -> void:
    recording.clear()
    velocity = Vector2.ZERO
    global_position = get_node("../Level/SpawnPoint").global_position
```

재생 (ghost.gd): 고스트는 `AnimatableBody2D` (sync_to_physics 켬)로 만든다. 위치를 직접 옮겨도 위에 선 플레이어가 같이 움직이고, 밟을 수 있는 실체가 된다. 이동 전에 `test_move` 로 막혔는지 확인하는 게 패러독스 판정이다.

```
# ghost.gd
extends AnimatableBody2D

signal collapsed
```

```
var frames: Array[Dictionary] = []
var tick := 0
var alive := true

func setup(data: Array[Dictionary]) -> void:
    frames = data
    add_to_group("resettable")
    reset()

func reset() -> void:
    tick = 0
    alive = true
    visible = true
    if frames.size() > 0:
        global_position = frames[0].pos

func _physics_process(_delta: float) -> void:
    if not alive or tick >= frames.size():
        return
    var f: Dictionary = frames[tick]
    var motion: Vector2 = f.pos - global_position
    if test_move(global_transform, motion):
        alive = false
        visible = false  # 여기에 글리치 연출
        collapsed.emit()
        return
    global_position = f.pos
    if f.interact:
        pass  # 겹쳐 있는 레버·버튼의 interact() 호출
    tick += 1
```

루프 (loop_manager.gd): 시간은 초가 아니라 물리 틱으로 센다. 프레임이 떨어져도 녹화와 재생이 어긋나지 않는다.

```
# loop_manager.gd
extends Node

const GhostScene := preload("res://ghost.tscn")

@export var loop_seconds := 10.0
@export var max_ghosts := 2

@onready var player := $"../Player"
@onready var ghost_root := $"../Ghosts"
```

```
var tick := 0

func _physics_process(_delta: float) -> void:
    tick += 1
    if tick >= int(loop_seconds * Engine.physics_ticks_per_second):
        end_loop()

func _unhandled_input(event: InputEvent) -> void:
    if event.is_action_pressed("end_loop"):
        end_loop()

func end_loop() -> void:
    if ghost_root.get_child_count() < max_ghosts:
        var g := GhostScene.instantiate()
        ghost_root.add_child(g)
        g.setup(player.recording.duplicate())
    tick = 0
    get_tree().call_group("resettable", "reset")
```

Player와 모든 레벨 오브젝트(문, 레버, 발판, 상자)는 `resettable` 그룹에 넣고 `reset()` 에서 초기 상태로 돌아가게 한다.

결정론 체크리스트

- [ ] 모든 게임 로직은 `_process` 가 아니라 `_physics_process` 에서
- [ ] 랜덤 요소 없음 (연출용 파티클은 예외)
- [ ] 레벨 오브젝트는 전부 `reset()` 으로 완전 초기화
- [ ] 압력판은 `Area2D` 의 겹친 바디 수로 판정해서, 플레이어와 고스트를 같은 방식으로 처리
- [ ] 위치 녹화 특성상 이동 발판 위에 있던 고스트는 발판이 안 움직여도 허공을 걷는다. 의도된 "유령" 느낌으로 두되, 레벨에서 이 상황이 퍼즐 답이 되지 않게 배치

세로 화면 설정: 프로젝트 설정 > Display > Window에서 뷰포트 720×1280, Stretch Mode `canvas_items`, Aspect `keep`. 렌더러는 웹·모바일 호환을 위해 처음부터 Compatibility로 둔다.

리플레이 저장 = 릴스 촬영 장치: 클리어할 때 모든 고스트의 녹화 데이터를 JSON으로 저장한다. 이걸 불러오는 리플레이 씬이 있으면, 직접 다시 플레이하지 않아도 같은 장면을 몇 번이든 똑같이 다시 촬영할 수 있다. 정답편 릴스, 별점 기록, 클리어 리플레이가 전부 이 하나로 해결된다.

```
# replay_io.gd (autoload)
extends Node

func save_run(stage_id: String, runs: Array) -> void:
    var data := []
    for frames in runs:
        var arr := []
        for f in frames:
```

```
            arr.append([f.pos.x, f.pos.y, f.flip, f.interact])
        data.append(arr)
    var file := FileAccess.open("user://replays/%s.json" % stage_id,
FileAccess.WRITE)
    file.store_string(JSON.stringify(data))

func load_run(stage_id: String) -> Array:
    var file := FileAccess.open("user://replays/%s.json" % stage_id,
FileAccess.READ)
    var runs := []
    for arr in JSON.parse_string(file.get_as_text()):
        var frames: Array[Dictionary] = []
        for v in arr:
            frames.append({"pos": Vector2(v[0], v[1]), "flip": v[2],
"interact": v[3]})
        runs.append(frames)
    return runs
```

`user://replays` 폴더는 첫 실행 때 `DirAccess.make_dir_recursive_absolute("user://replays")`로 만들어둔다.

촬영 자동화 (Movie Maker 모드): 고닷 4에 내장된 Movie Maker 모드는 실제 속도와 상관없이 고정 프레임으로 영상을 뽑아준다. 리플레이 씬을 Movie Maker로 실행하면 프레임 드랍 없는 깨끗한 60fps 영상이 나오고, 이걸 insta_auto 파이프라인에 넣어 자막·업로드까지 연결한다.

릴스 모드 토글 (디버그 키 F9): 조작 버튼과 패널을 숨기고, 상단에 텍스트 한 줄 슬롯("10초 전의 나 × 4")을 띄우고, 카메라를 방에 꽉 맞춘다. 게임 화면과 릴스 화면을 같은 코드로 관리한다.

릴스 콘텐츠 전략

게임을 다 만들고 홍보하는 게 아니라, 게임 자체를 릴스 소재가 계속 나오는 공장으로 만든다. 1주차부터 주 3회, 6주 동안 약 18개를 올린다.

계정: 기존 ZUN(@zun_it_) 계정에서 시리즈로 시작한다. "03년생 대학생의 AI·개발 성장기"를 보러 온 팔로워에게 "AI랑 첫 게임 만들기"는 결이 맞고, 0명에서 시작하는 새 계정보다 초반 노출이 유리하다. 출시 후 게임 클립 반응이 개발 일지보다 확실히 좋으면 그때 게임 전용 계정을 따로 연다.

시리즈 이름: "10초 전의 나" (EP.0, EP.1 …). 게임 이름보다 규칙이 바로 떠오르는 이름이 썸네일과 첫 줄 텍스트에서 더 잘 먹힌다.

포맷	길이	구성	노리는 반응	빈도
퍼즐 챌린지 (문제 편)	10~15초	방 전체를 보여주고 "고스트 2명으로 깰 수 있을까?" → 실패하는 시도 → "정답은 다음 릴스"	댓글, 저장	주 1

포맷	길이	구성	노리는 반응	빈도
정답편·클리어 리플레이	15~20초	모든 고스트가 맞물리는 순간부터 시작 → 루프별로 분해 → 마지막 프레임이 첫 장면으로 이어짐	반복 시청, 공유	주 1
개발 일지	20~30초	결과 화면부터 → "Claude랑 고닷으로 만듦" → 원리 하나만 설명 → 댓글 키워드로 코드 배포	팔로우, 저장	주 1
패러독스 실패 모음	7~10초	과거의 나를 가둬서 붕괴하는 장면만 연속, 글리치 효과음	공유	격주
플레이테스트 반응	15초	친구가 막히다가 깨는 순간	공유	가끔

첫 1초 규칙

- 빈 방으로 시작하지 않는다. 첫 프레임부터 고스트가 3명 이상 움직이고 있어야 한다.
- 상단 텍스트는 한 줄, 숫자로: "10초 전의 나 × 4", "과거의 나 3명으로 탈출".
- 영상 끝은 루프 리셋 장면. 게임의 테마가 루프라서 영상도 루프로 이어지고, 반복 시청이 자연스럽게 늘어난다.
- 되감기 효과음으로 시작한다. 시리즈 시그니처 사운드.

댓글 키워드 (기존 방식 그대로): "루프" → 출시 알림과 플레이 링크 DM, "고닷" → 고스트 녹화 코드 배포. 퍼즐 챌린지에는 "정답을 댓글로"를 붙이고, 가장 먼저 맞힌 사람 아이디를 정답편 화면에 넣는다. 참여한 사람이 정답편을 공유할 이유가 생긴다.

제작 흐름

1. 클리어할 때 리플레이 JSON 자동 저장
2. 리플레이 씬 + 릴스 모드(F9)를 Movie Maker로 렌더
3. insta_auto로 상단 텍스트·자막·업로드
4. 성과는 기존 인스타 성과 트래킹 시트에 포맷 값 "에코루프-챌린지 / 리플레이 / 개발일지 / 실패모음"으로 기록

판단 기준: 2주 단위로 포맷별 평균 조회수, 저장+공유, 신규 팔로워를 비교한다. 다른 포맷의 절반에 못 미치는 포맷은 빈도를 줄이고 잘되는 포맷에 몰아준다.

개발 일정과 MVP 범위 (6주, 공개 개발)

4주에서 6주로 늘렸다. 개발만 하면 4주면 되지만, 주 3회 릴스 촬영·편집을 같이 굴리면 매주 시간의 3분의 1 정도가 콘텐츠로 빠진다. 개발은 공강인 월요일과 주말에 몰고, 짧은 평일 저녁은 촬영·편집·업로드에 쓴다. 가장 중요한 건 여전히 1주차 끝의 재미 검증이다. 회색 박스 방 하나가 재밌지 않으면 규칙부터 다시 본다.

1주차부터 공개 개발: 6주 동안 개발과 릴스를 나란히 굴린다



6주 공개 개발 로드맵 · 개발과 릴스 2트랙

1주차 할 일

- [ ] Godot 4 프로젝트 생성, 입력 맵 등록 (move_left, move_right, jump, interact, end_loop, undo_ghost)
- [ ] Player 이동·점프
- [ ] LoopManager 틱 카운트와 리셋
- [ ] 녹화 → Ghost 재생
- [ ] 압력판 + 문으로 1-2 스테이지 회색 박스 버전
- [ ] 친구 1명에게 설명 없이 해보게 하기

MVP에 넣는 것: 12~15스테이지, 오브젝트 8종, 세로 9:16 화면, 모바일 터치 조작, 고스트 개별 재녹화, 루프 타임라인, 별점, 되감기·글리치 연출, 클리어 리플레이 저장, 릴스 모드, 스테이지 선택 화면, 웹 빌드.

MVP에서 빼는 것: 레벨 에디터, 스토리 컷신, 세부 설정 메뉴, 온라인 랭킹. 클리어 기록과 별점만 브라우저에 저장하고, 랭킹은 릴스 댓글과 리그램 이벤트로 대신한다.

확장 아이디어와 리스크

가장 큰 리스크는 기술이 아니라 레벨 디자인이다. 시스템은 1~2주면 돌아가지만, "답이 하나로 떨어지는 좋은 퍼즐" 15개를 만드는 데 시간이 가장 많이 든다.

리스크	이런 신호가 보이면	대응
퍼즐이 너무 쉽거나 답이 여러 개	테스터가 의도와 다른 방법으로 쉽게 깸	고스트 상한을 줄이거나 루프 시간을 줄여 우회로를 막는다
녹화와 재생이 미세하게 어긋남	같은 플레이인데 고스트 위치가 몇 픽셀씩 다름	로직을 전부 _physics_process로 옮기고, 틱 단위로만 시간을 센다
상자 물리가 고스트와 꼬임	상자가 고스트에 끼어 튕겨 나감	상자를 MVP에서 빼고 3-3을 다른 퍼즐로 교체
폰 브라우저에서 안 돌아감	아이폰 사파리에서 소리가 안 나거나 화면이 깨짐	2주차에 폰 테스트를 일정에 고정, Compatibility 렌더러 유지, 첫 터치 후 오디오 시작
릴스 조회수가 안 나옴	첫 3개 릴스가 기존 계정 평균에 못 미침	첫 1초 장면과 상단 문구만 바꿔 같은 클립을 다시 올리고, 개발 일지보다 퍼즐 챌린지 비중을 늘린다
개발 일지와 게임 클립 반응이 갈림	개발 일지는 팔로우, 게임 클립은 조회만 높음	정상이다. 게임 클립으로 노출을 얻고, 고정 게시물·하이라이트에 개발 일지를 모아 팔로우로 바꾼다
일정 초과	5주차 끝에 10스테이지 미만	12스테이지 컷 기준대로 줄이고, 연출과 출시 릴스를 우선한다

MVP 이후에 붙일 수 있는 것

- 최소 루프 기록: 스테이지별 "최소 고스트 수"와 "최단 시간" 랭킹. 같은 퍼즐을 더 적은 나로 푸는 재도전 동기
- 레벨 에디터 + 공유 코드: 플레이어가 만든 방을 문자열로 공유
- 2인 실시간 모드: 각자 루프를 돌며 서로의 고스트와 협력
- 적이 고스트를 쫓는 챕터: 적은 "가장 가까운 나"를 추적하고, 과거의 나를 미끼로 쓴다
- 클리어 리플레이를 GIF·영상으로 저장하는 버튼: 홍보용 숏폼 소재로 바로 쓸 수 있다